Written Discussion (roughly one page, as required)

The full results table and sweep plots are in results_1_3.pdf and figures/; this page is the written discussion only.

All three disturbances increase release and/or completion jitter. Seven configurations kept the deadline-hit fraction at 95-100% while release-jitter std grew well past the ~0.03 ms control baseline: background load at duty 0.50 (std 1.74 ms) and 0.75 (1.99 ms), delay probability 0.05/0.10/0.20 at 5 ms (std 1.01/1.22/2.12 ms), and delay magnitude 5 ms/8 ms at p = 0.10 (std 1.32/1.95 ms). The clearest case is p = 0.20 at 5 ms (release-jitter std 2.12 ms, 100% hit) against the control (std 0.03 ms, 100% hit): identical hit fraction, very different regularity. This confirms Section 1.3's point that meeting every deadline and having low jitter are independent properties.

Disturbance	Section 1.6 source approximated	Where the analogy breaks down
A: background load thread	OS jitter / general interference from competing tasks.	CPython threads share one interpreter lock, quantizing delay to ~5 ms switch intervals; a real RTOS uses priorities and preemption.
B: random start delay	Interrupts / DMA stalls delaying a task's start.	It is a sleep (CPU stays free), whereas a real interrupt or DMA transfer consumes CPU or bus time; real interrupts are often periodic or bursty.
C: exec-time variability	Cache effects and pipeline hazards.	Drawn from an i.i.d. distribution; real variability is state-dependent and correlated (warm/cold cache, branch history), not independent per call.